

Designing a Secure, Highly Available Cloud Solution for A Medical Company

ITC 6480 — Cloud Architecting Final Project ()

Wisdom Djam

M.P.S. Informatics — Information Security Management
Northeastern University

June 16, 2026

A multi-AZ, three-tier AWS architecture for a medical SaaS platform

AWS Architecture: A Medical Company

Wisdom Djam

M.P.S. Informatics — Information Security Management
Northeastern University

June 16, 2026

A multi-AZ, three-tier AWS architecture for a medical SaaS platform

- **Open:** “My name is Wisdom Djam. I’ll present my proposed AWS cloud architecture for ‘A Medical Company,’ a medical SaaS startup that is about to launch publicly and wants to migrate off leased physical servers.”
- **Set the stakes early:** This is a *medical* company — the data is sensitive personal health information, the launch is imminent, and they expect rapid growth. I’ll keep coming back to those three facts because they drive every decision I made.
- **Roadmap the talk:** “I’ll start with the problem, give a high-level summary of my solution, then walk through the architecture and show how it meets their security, availability, and auditing requirements, and finish with my recommended next steps.”
- **Time check:** 20–25 seconds on this slide.
- **Transition:** “Business problem.”

Agenda

- ➊ Introduction — Problem & Solution
- ➋ Executive Summary
- ➌ Requirements
- ➍ Assumptions
- ➎ Architecture Diagram
- ➏ Network & Security
- ➐ Scalability, HA & Business Continuity
- ➑ Monitoring & Auditing
- ➒ Next Steps & Conclusion

Author

Wisdom Djam

M.P.S. Informatics

(Information Security Management)

Project scope

An individual, design-only proposal — worksheet-driven and aligned to the AWS Well-Architected Framework. No production build.

AWS Architecture: A Medical Company

└ Agenda

Agenda	
❶ Introduction — Problem & Solution ❷ Executive Summary ❸ Requirements ❹ Assumptions ❺ Architecture Diagram ❻ Network & Security ❼ Scalability, HA & Business Continuity ❽ Monitoring & Auditing ❾ Next Steps & Conclusion	Author Wisdom Djani M.P.S. Informatics (Information Security Management) Project scope An individual, design-only proposal — worksheet-driven and aligned to the AWS Well-Architected Framework. No production build.

- **Purpose:** Give the audience a map of the talk and satisfy the rubric's "agenda + who worked on the project." Since this is an individual project, I note that I completed all sections myself.
- **Say:** Run through the nine items quickly — one breath. "I worked through each of these areas myself, translating the customer's requirements into an AWS design."
- **Reinforce scope:** "This is a design proposal grounded in the AWS Well-Architected Framework — its five pillars of operational excellence, security, reliability, performance efficiency, and cost optimization. I am recommending an architecture rather than building a live environment."
- **Why name the framework:** It shows the instructor my choices are anchored in a recognized standard rather than being ad hoc.
- **Transition:** "Problems and Solutions"

The Problem

- A Medical Company is a software-as-a-service medical startup. Its platform connects patients and doctors for online appointments, remote consultation/diagnosis, electronic prescription transfer, payments, and document/image processing, serving users across APAC, the US, and Europe.
- Its current development/test environment runs on **leased physical Windows servers** (IIS web & application tiers, Microsoft SQL Server backend) at a hosting provider — not built for the imminent public launch or for rapid growth.
- The application stores **highly sensitive medical PII**, which demands strict access control, encryption, and complete auditability.

My Solution

- A **multi-AZ, three-tier AWS architecture** that mirrors the customer's existing topology while adding elasticity, high availability, defense-in-depth security, and full CloudTrail auditing — with fully isolated Development/Test and Production networks.

AWS Architecture: A Medical Company

└ Introduction: Problem & Solution

Introduction: Problem & Solution	
The Problem	- A Medical Company is a software-as-a-service medical startup. Its platform connects patients and doctors for online appointments, remote consultation/diagnosis, electronic prescription transfer, payments, and document/image processing, serving users across APAC, the US, and Europe. - Its current development/test environment runs on leased physical Windows servers (IIS web & application tiers, Microsoft SQL Server backend) at a hosting provider — not built for the imminent public launch or for rapid growth. - The application stores highly sensitive medical PII, which demands strict access control, encryption, and complete auditability.
My Solution	A multi-AZ, three-tier AWS architecture that mirrors the customer's existing topology while adding elasticity, high availability, defense-in-depth security, and full CloudTrail auditing — with fully isolated Development/Test and Production networks.

- **Lead with business, not technology:** The customer cares about launching safely and growing, not about EC2. “A Medical Company built a platform that connects patients and doctors — appointments, remote diagnosis, prescriptions, payments, and document processing — to about three continents.”
- **Precision:** “Today they run on rented physical Windows servers. That can't scale for launch-day demand, it has single points of failure, and it wasn't designed for the compliance.”
- **Data Sensitivity:** I emphasize medical PII because it's the thread that justifies my security and auditing sections later. This is effectively HIPAA territory,.
- **Thesis:** The “My Solution” sentence is the one-line summary of the entire presentation. I pause on the four pillars — elasticity, high availability,

What I set out to do

- Translate the seven customer requirements into a concrete AWS reference architecture.
- Preserve the existing three-tier topology while removing every single point of failure.

How I accomplished it

- **Compute:** EC2 (Windows) web & app tiers in Auto Scaling Groups across two AZs.
- **Database:** Amazon RDS for SQL Server, Multi-AZ.
- **Network:** Two VPCs (Dev/Test & Prod); public ELB subnets, private app/DB subnets.
- **Identity:** IAM groups, password policy, MFA, app→S3 roles.
- **Resilience:** ELB + Auto Scaling self-healing; Multi-AZ DB failover.
- **Governance:** CloudTrail + CloudWatch + a secured S3 log bucket.

└ Executive Summary

Executive Summary

What I set out to do

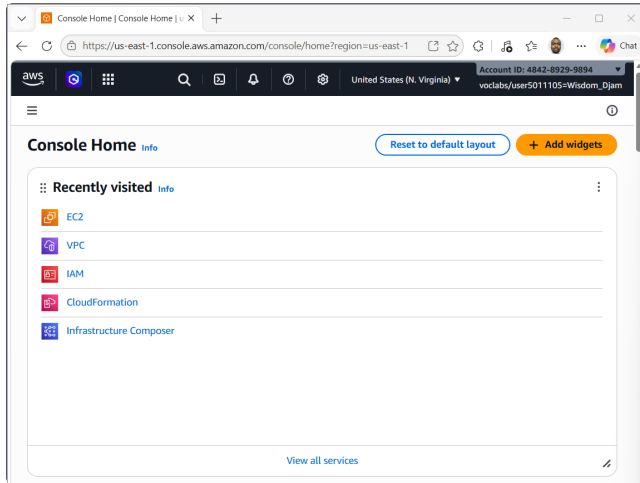
- Translate the seven customer requirements into a concrete AWS reference architecture.
- Preserve the existing three-tier topology while removing every single point of failure.

How I accomplished it

- **Compute:** EC2 (Windows) web & app tiers in Auto Scaling Groups across two AZs.
- **Database:** Amazon RDS for SQL Server, Multi-AZ.
- **Network:** Two VPCs (Dev/Test & Prod): public ELB subnets, private app/DB subnets.
- **Identity:** IAM groups, password policy, MFA, app-S3 roles.
- **Resilience:** ELB + Auto Scaling self-healing; Multi-AZ DB failover.
- **Governance:** CloudTrail + CloudWatch + a secured S3 log bucket.

- **Framing:** “If there is one slide to remember, this is it — it is my whole design in headlines.”
- **What/How structure:** First I state the goal — turn the requirements into an architecture, keep their three-tier topology, and remove single points of failure. Then I walk the six “how” lines: compute, database, network, identity, resilience, governance — one sentence each, because the detail comes in later sections.
- **Tie each line to a requirement:** compute + Auto Scaling = “double the servers”; RDS Multi-AZ = business continuity; two VPCs = environment separation; IAM/MFA = access best practices; ELB + ASG = resilience; CloudTrail = auditing. This proves the design is requirement-driven, not arbitrary.
- **Transition:** The requirements those choices are built on follow next.

Working Environment — AWS Console

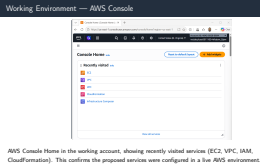


AWS Console Home in the working account, showing recently visited services (EC2, VPC, IAM, CloudFormation). This confirms the proposed services were configured in a live AWS environment.

AWS Architecture: A Medical Company

└ Working Environment — AWS Console

- This full-page capture anchors that the design was built and verified in a real AWS account, not described in the abstract.
- If asked, I can navigate to any of these services and show the specific configuration on the slides that follow.



Requirements

Taken directly from the customer's high-level and detailed requirements:

- ➊ **Access permissions** per AWS best practices (least privilege, MFA, password policy).
- ➋ **Best-practice networks** providing all needed services across environments.
- ➌ Match the **current three-tier design**; **double the servers** on demand.
- ➍ **Secure all medical PII** (encryption, isolation, controlled access).
- ➎ **Load balancers** for web & app tiers supporting HTTP, HTTPS, and TCP.
- ➏ Architecture is **resilient** / built for business continuity.
- ➐ **Auditing** that tracks all user actions.

Network constraints

- No external access to app or DB tiers
- Minimize IP usage (shrink attack surface)
- Separate Dev/Test from Production
- Web ELB receives Internet traffic on 443
- Each tier accepts traffic only from the tier in front of it, on 443

AWS Architecture: A Medical Company

└ Requirements

Requirements	
Taken directly from the customer's high-level and detailed requirements:	
- Access permissions per AWS best practices (least privilege, MFA, password policy). - Best-practice networks providing all needed services across environments. - Match the current three-tier design: double the servers on demand. - Secure all medical PHI (encryption, isolation, controlled access). - Load balancers for web & app tiers supporting HTTP, HTTPS, and TCP. - Architecture is <i>resilient</i> / built for business continuity. - Auditing that tracks all user actions.	Network constraints - No external access to app or DB tiers - Minimize IP usage (shrink attack surface) - Separate Dev/Test from Production - Widespread ELB receives Internet traffic on 443 - Each tier accepts traffic only from the tier in front of it, on 443

- **Why this slide exists:** These seven requirements are the rubric I hold my own design to — every later slide maps back to one of them, and I'll point that out as I go.
- **Read with intent — group them:** requirements 1 and 4 are *security*; 2 and 5 are *network and connectivity*; 3 and 6 are *scalability and continuity*; 7 is *governance*. Grouping shows I understand the themes, not just the list.
- **Spotlight the constraints box:** These are the most technically specific demands. “No external access to the app or database tiers” and “each tier accepts traffic only from the tier in front of it, on 443” are exactly what produced my security-group chaining design — I preview that here so the Network slide lands.
- **Explain “minimize IP usage” simply:** right-sizing the subnets means fewer exposed addresses, which is a smaller attack surface — good security

Assumptions

- The application is **compatible with AWS-hosted Windows Server 2016** and Amazon RDS for SQL Server Standard, allowing a lift-and-shift migration with minimal refactoring.
- A **single AWS Region** (e.g., US East) is acceptable for the initial launch; multi-Region disaster recovery is a planned later phase.
- **Two Availability Zones** provide sufficient high availability for launch.
- The customer holds (or will obtain) a **TLS/SSL certificate** via AWS Certificate Manager for HTTPS termination at the load balancer.
- The requirement to “**double the servers**” is satisfied by Auto Scaling *maximum capacity*, not manual provisioning.
- For the initial design, **security, availability, and compliance take priority over cost**; cost optimization is a follow-up phase.
- Because the platform stores medical data, the design uses **HIPAA-eligible AWS services** and assumes a signed AWS Business Associate Addendum (BAA).
- The DBAs need standard access only — **no advanced RDBMS configuration** — which makes a managed database (RDS) the natural fit.

AWS Architecture: A Medical Company

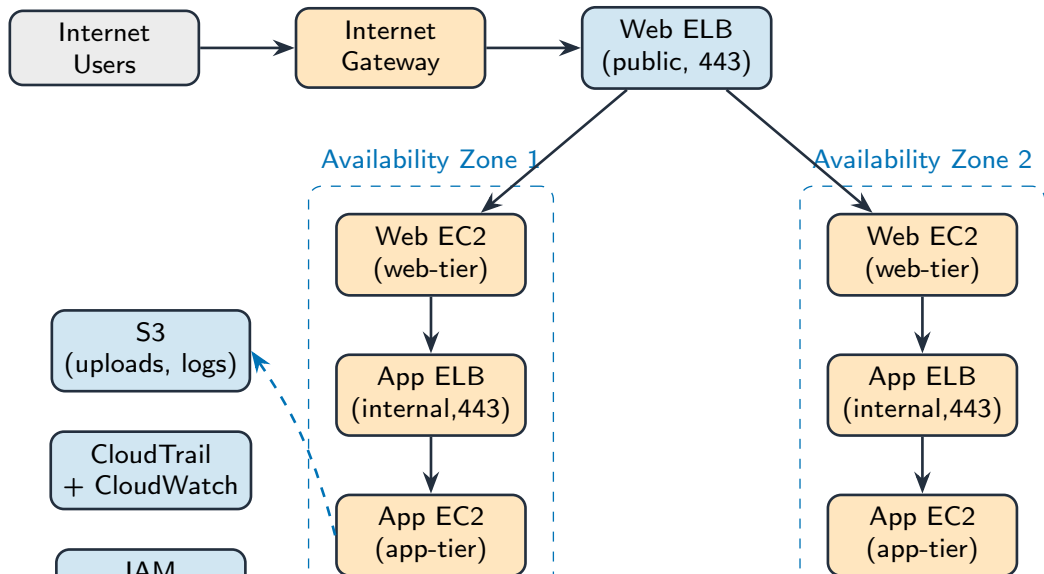
└ Assumptions

Assumptions

- The application is **compatible with AWS-hosted Windows Server 2016** and Amazon RDS for SQL Server Standard, allowing a lift-and-shift migration with minimal refactoring.
- A **single AWS Region** (e.g., US East) is acceptable for the initial launch; multi-Region disaster recovery is a planned later phase.
- **Two Availability Zones** provide sufficient high availability for launch.
- The customer holds (or will obtain) a **TLS/SSL certificate** via AWS Certificate Manager for HTTPS termination at the load balancer.
- The requirement to **"double the servers"** is satisfied by Auto Scaling maximum capacity; not manual provisioning.
- For the initial design, **security, availability, and compliance take priority over cost**; cost optimization is a follow-up phase.
- Because the platform stores medical data, the design uses **HIPAA-eligible AWS services** and assumes a signed AWS Business Associate Addendum (BAA).
- The DBAs need standard access only — **no advanced RDBMS configuration** — which makes a managed database (RDS) the natural fit.

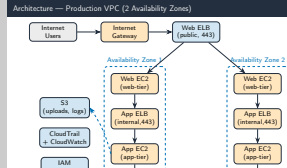
- **Why assumptions matter:** They make my design defensible — they state what I took as given, so the design is judged on its own terms. “Here is what I assumed; if any of these change, the design adapts.”
- **Walk the important ones:** lift-and-shift Windows/SQL means I’m matching their existing stack rather than forcing a rewrite; single Region now with multi-Region DR later is a deliberate scoping decision; two AZs give enough HA for launch (this also sets up Role-Play #2 about single-AZ deployments); “double the servers” is an Auto Scaling maximum, i.e., elasticity rather than buying hardware.
- **Compliance assumption:** I call out HIPAA-eligible services and an AWS BAA because medical data carries legal weight, not just technical requirements — this shows I’m thinking about the customer’s real risk.
- **Managed-DB rationale:** the customer said no advanced RDBMS

Architecture — Production VPC (2 Availability Zones)



AWS Architecture: A Medical Company

Architecture — Production VPC (2 Availability Zones)



- This is my most important slide — I'll spend the most time here, because diagrams are weighted heavily.
- Walk the request path along the arrows: "A patient's request comes from the Internet, through the Internet Gateway, to my *public* Web ELB on port 443. The ELB distributes traffic to Web EC2 instances that live in *private* subnets across two Availability Zones."
- **Continue inward:** "Each web server talks to an internal Application ELB that fronts the App EC2 instances. The application tier is the only thing permitted to reach the database."
- **Point at the green dashed line:** "The database is Amazon RDS for SQL Server running Multi-AZ — a synchronous standby in the second AZ. If the primary's AZ fails, RDS fails over automatically."
- **Point at the left-side services:** "S3 stores uploaded documents and..."

Services / techniques

- **Two VPCs:** Production and Dev/Test (full isolation).
- Public subnets (ELB only) + private subnets (Web, App, DB) in each AZ.
- **Security groups** (stateful firewalls) chained tier-to-tier on **port 443 only**.
- No IGW route for the app/DB tiers ⇒ **no external access**.
- **IAM:** 3 groups (SysAdmin, DBA, Monitoring), least-privilege policies, **MFA**, and an app→S3 **role**.
- **Password policy:** 8+ chars, upper/lower/number/special, 90-day rotation, no reuse of last 3.
- **Encryption:** TLS in transit (ACM); at rest via KMS (EBS, RDS, S3).

Why I chose this

- SG chaining = **defense in depth**; each tier trusts only the tier in front of it.
- Private tiers + a single public entry point shrink the **attack surface** for PII.
- Separate VPCs stop a Dev/Test issue from ever reaching production patient data.
- IAM groups, MFA, and roles (not embedded keys) follow least-privilege best practice.
- Encryption everywhere protects medical PII in transit and at rest.

AWS Architecture: A Medical Company

└ Network & Security

Network & Security	
Services / techniques	Why I chose this
- Two VPCs: Production and Dev/Test (full isolation). - Public subnets (ELB only) + private subnets (Web, App, DB) in each AZ. - Security groups (stateful firewalls) chained tie-to-tie on port 443 only. - No IGW route for the app/DB tiers => no external access. - IAM: 3 groups (SysAdmin, DBA, Monitoring), least-privilege policies, MFA, and an app->S3 role. - Password policy: 8+ chars, upper/lower/number/special, 90-day rotation, no reuse of last 3. - Encryption: TLS in transit (ACM); at rest via KMS (ERS, RD25, S3).	- SG chaining => defense in depth; each tier trusts only the tier in front of it. - Private tiers + a single public entry point shrink the attack surface for PII. - Separate VPCs stop a Dev/Test issue from ever reaching production patient data. - IAM groups, MFA, and roles (not embedded keys) follow least-privilege best practice. - Encryption everywhere protects medical PII in transit and at rest.

- This is the compliance heart of the design — the area most relevant to my Information Security Management concentration.
- **Headline:** The database has *no* inbound rule from the Internet. It accepts traffic only from the application-tier security group, and only on port 443.
- **SG chaining, simply:** Each tier trusts only the tier directly in front of it — the web SG accepts from the public ELB, the app SG accepts from the web SG, the DB SG accepts from the app SG. That is defense in depth at the network layer.
- **Two VPCs, not one:** Dev/Test is isolated from Production in a separate VPC so test activity can never touch live patient data, and a misconfiguration in one environment cannot cascade into the other.
- **Identity (maps to the User Authentication worksheet and Requirement #1):** three IAM groups with least-privilege policies; MFA

Network & Security — Security Group Chaining

The screenshot shows the AWS Management Console for a VPC in the us-east-1 region. The left sidebar contains navigation links for VPC, Security Groups, and PrivateLink and Lattice. The main content area displays the details for the security group 'sg-067e09ae88a6614ef - db-tier-sg'. A green notification bar at the top indicates that inbound security group rules were successfully modified. The details section shows the security group name, ID, description, owner, and rule counts. Below this, the 'Inbound rules' tab is selected, showing a single rule with ID 'sgr-042872d70dbdb36fc' that allows HTTPS traffic on port 443 from the 'app-tier-sg' security group.

sg-067e09ae88a6614ef - db-tier-sg

Details

Security group name db-tier-sg	Security group ID sg-067e09ae88a6614ef	Description Database tier - accepts 443 from app tier only	VPC ID vpc-053ce8e545678a5c3
Owner 484289299894	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

Inbound rules (1)

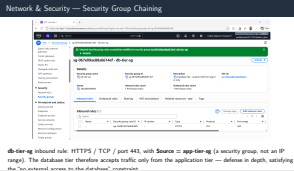
Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-042872d70dbdb36fc	-	HTTPS	TCP	443

db-tier-sg inbound rule: HTTPS / TCP / port 443, with **Source** = **app-tier-sg** (a security group, not an IP range). The database tier therefore accepts traffic only from the application tier — defense in depth, satisfying

AWS Architecture: A Medical Company

└ Network & Security — Security Group Chaining

- The key control here is the *source*: a security group reference, not a CIDR block. That is what proves the database is unreachable from the Internet and trusts only the app tier.
- This was built live in the Learner Lab (security-group creation and editing were permitted), so it is a real configuration, not a mock-up.



Network & Security — IAM Password Policy

aws Edit password policy | IAM | Global

https://us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/account_settings/edit_password

Account ID: 4842-8929-9894
voclabs/user5011105-Wisdom_Djam

Account Settings > Edit password policy

Password policy

☐ IAM default
Apply default password requirements.

☒ Custom
Apply customized password requirements.

Password minimum length.
Enforce a minimum length of characters.

characters
Needs to be between 6 and 128.

Password strength

- ☒ Require at least one uppercase letter from the Latin alphabet (A-Z)
- ☒ Require at least one lowercase letter from the Latin alphabet (a-z)
- ☒ Require at least one number
- ☒ Require at least one non-alphanumeric character (!@#\$%^&*()_+-=[]{}|'')

Other requirements

- ☒ Turn on password expiration
 - Expire password in day(s)
Needs to be between 1 and 1095 days.
 - ☐ Password expiration requires administrator reset
 - ☒ Allow users to change their own password
 - ☒ Prevent password reuse
 - Remember password(s)
Needs to be between 1 and 24.

Custom IAM account password policy: 8-character minimum, all four complexity classes, 90-day expiration, and no reuse of the last three passwords — meeting the customer's User Authentication requirement.

AWS Architecture: A Medical Company

└ Network & Security — IAM Password Policy

- This satisfies Requirement #1 (access permissions per best practice) and the User Authentication worksheet.
- **Honest note:** In the AWS Academy Learner Lab the assumed role lacks `iam:UpdateAccountPasswordPolicy`, so the policy is shown as configured rather than committed. In a production account with that permission it would be saved. This also demonstrates how IAM permission boundaries constrain a role.



AWS Architecture: A Medical Company

└ Network & Security — IAM Password Policy



- **This is the compliance heart of my talk — the section I care most about, given my Information Security Management concentration.**
- **Open with the headline:** “The database has *no* inbound rule from the Internet. It accepts traffic only from the application-tier security group, and only on port 443.”
- **Explain security-group chaining simply:** “Each tier only trusts the tier directly in front of it — the web SG accepts from the public ELB, the app SG accepts from the web SG, and the DB SG accepts from the app SG. That’s defense in depth at the network layer.”
- **Two VPCs, not one:** “I isolate Dev/Test from Production in separate VPCs so test activity can never touch live patient data, and a misconfiguration in one environment cannot cascade into the other.”
- **Identity story (this maps to the User Authentication worksheet and**

Services / techniques

- **Auto Scaling Groups** for the Web (WebTier) & App (AppTier) tiers across 2 AZs.
- **Elastic Load Balancing** distributes traffic and runs health checks.
- An unhealthy instance (OS or app unresponsive) is **automatically terminated and replaced**.
- **RDS Multi-AZ** synchronous standby with automatic failover.
- ASG maximum capacity sized to **double the servers** on demand.

Why I chose this

- Two AZs remove single-AZ failure as a single point of failure.
- Auto Scaling = elasticity: pay for what I need; absorb launch spikes and growth.
- ELB + ASG together = self-healing (the business-continuity requirement).

Design-specified.

Auto Scaling Group

(WebTier/AppTier): Min 2 / Desired 2 / Max 6 across two AZs. The Learner Lab role lacks `ec2:CreateLaunchTemplate`, so the ASG is specified in the design (see architecture diagram) rather than built live.

Design-specified.

Amazon RDS Multi-AZ (SQL Server): synchronous standby in AZ2 with automatic failover — specified per the business-continuity requirement.

AWS Architecture: A Medical Company

└ Scalability, High Availability & Business Continuity

Scalability, High Availability & Business Continuity Services / techniques • Auto Scaling Groups for the Web (WebTier) & App (AppTier) (one across 2 AZs). • Elastic Load Balancing distributes traffic and runs health checks. • An unhealthy instance (OS or app unresponsive) is automatically terminated and replaced. • RDS Multi-AZ synchronous standby with automatic failover. • ASG maximum capacity sized to <i>double the servers</i> on demand. Why I chose this • Two AZs remove single-AZ failure as a single point of failure. • Auto Scaling = elasticity: pay for what I need; always launch spikes and growth. • ELB + ASG together is self-healing (the business-continuity requirement).	Design-specified. Auto Scaling Group (WebTier / AppTier): Min 2 / Desired 2 / Max 6 across two AZs. The Learner Lab rule lacks az1:CreateAndConfigure, so the ASG is specified in the design (see architecture diagram) rather than built in. Design-specified. Amazon RDS Multi-AZ (SQL Server) synchronous standby in AZ2 with automatic failover — specified per the business-continuity requirement.
--	--

- **Distinguish load balancing from elasticity (Role-Play #3 — I should expect this question):** “Load balancing spreads today’s traffic across the servers I already have. Elasticity changes how many servers exist over time. The ELB does the first; Auto Scaling does the second. Together they handle both steady distribution and changing demand.”
- **Explain self-healing concretely:** “If a server stops responding, the ELB health check fails, the ELB stops routing traffic to it, and the Auto Scaling Group terminates it and launches a fresh replacement — with no human involved. That’s my business-continuity answer for the web and app tiers.”
- **Database durability:** “RDS Multi-AZ keeps a synchronous standby in the second Availability Zone and fails over automatically, so the database is not a single point of failure either.”
- **“Double the servers”:** I stress that this requirement is met simply by

Services / techniques

- **AWS CloudTrail:** records every API call / user action (console, CLI, SDKs).
- Trail → a **dedicated, access-restricted S3 bucket** with log-file validation enabled.
- **Amazon CloudWatch:** metrics, dashboards, and alarms (a CPU alarm triggers Auto Scaling).
- CloudWatch Logs for OS and application logs.
- The Monitoring IAM group has **read-only** visibility into EC2, S3, and RDS.

Why I chose this

- CloudTrail provides the full **audit trail** of all user actions (Requirement #7).
- A locked-down S3 log bucket keeps records **secure and tamper-evident**.
- CloudWatch alarms close the loop with Auto Scaling (performance + availability).
- A read-only Monitoring group enforces **separation of duties**.

AWS Architecture: A Medical Company

└ Monitoring & Auditing

Services / techniques

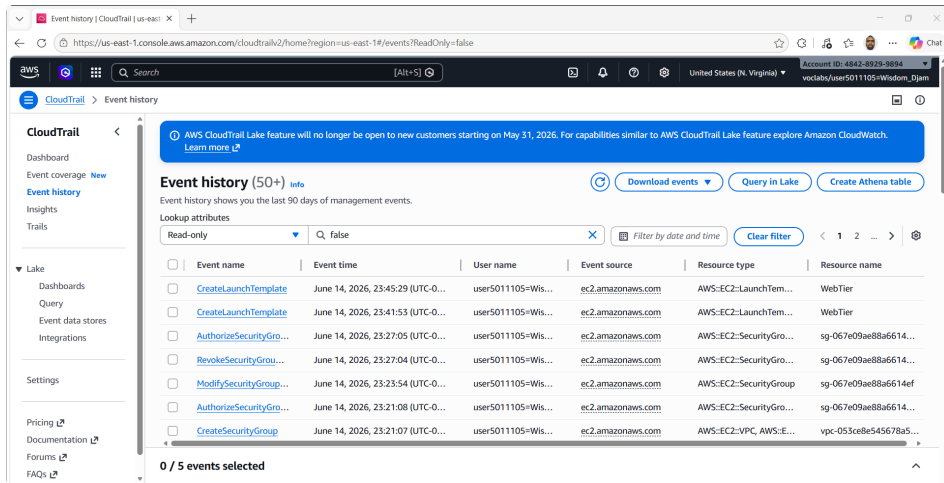
- **AWS CloudTrail**: records every API call / user action (console, CLI, SDKs).
- Trail → a **dedicated, access-restricted S3 bucket** with log-file validation enabled.
- **Amazon CloudWatch**: metrics, dashboards, and alarms (a CPU alarm triggers Auto Scaling).
- CloudWatch Logs for OS and application logs.
- The Monitoring IAM group has **read-only** visibility into EC2, S3, and RDS.

Why I chose this

- CloudTrail provides the **full audit trail** of all user actions (Requirement #7).
- A locked-down S3 log bucket keeps records **secure and tamper-evident**.
- CloudWatch alarms close the loop with Auto Scaling (performance + availability).
- A read-only Monitoring group enforces **separation of duties**.

- **Key distinction:** CloudTrail answers *who did what* (governance and audit); CloudWatch answers *how is the system performing* (operations and metrics). A medical company needs both, but the audit trail is the compliance must-have.
- **CloudTrail detail:** it logs every API call across the account and delivers the records to a dedicated S3 bucket; locking that bucket down and enabling log-file validation makes the trail tamper-evident.
- **Why a restricted bucket:** if the logs could be edited, the audit trail would be worthless. Protecting and validating them preserves integrity — a core information-security principle.
- **CloudWatch's role:** the CPU alarm is what triggers the Auto Scaling described earlier — monitoring and scaling are connected.
- **Separation of duties:** the Monitoring IAM group can see resources but

Monitoring & Auditing — CloudTrail Event History



The screenshot shows the AWS CloudTrail Event History console. The left sidebar contains navigation links for CloudTrail, Dashboard, Event coverage, Event history (selected), Insights, Trails, Lake, Dashboards, Query, Event data stores, Integrations, Settings, Pricing, Documentation, Forums, and FAQs. The main content area displays the Event history (50+) section, which shows the last 90 days of management events. A blue banner at the top indicates that the AWS CloudTrail Lake feature will no longer be open to new customers starting on May 31, 2026. Below the banner, there are buttons for Download events, Query in Lake, and Create Athena table. The event list is filtered by Read-only and false. The table shows the following events:

	Event name	Event time	User name	Event source	Resource type	Resource name
☐	CreateLaunchTemplate	June 14, 2026, 23:45:29 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::LaunchTem...	WebTier
☐	CreateLaunchTemplate	June 14, 2026, 23:41:53 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::LaunchTem...	WebTier
☐	AuthorizeSecurityGro...	June 14, 2026, 23:27:05 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGro...	sg-067e09ae88a6614...
☐	RevokeSecurityGrou...	June 14, 2026, 23:27:04 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGro...	sg-067e09ae88a6614...
☐	ModifySecurityGroup...	June 14, 2026, 23:23:54 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGroup	sg-067e09ae88a6614ef
☐	AuthorizeSecurityGro...	June 14, 2026, 23:21:08 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGro...	sg-067e09ae88a6614...
☐	CreateSecurityGroup	June 14, 2026, 23:21:07 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::VPC, AWS::E...	vpc-053ce8e545678a5...

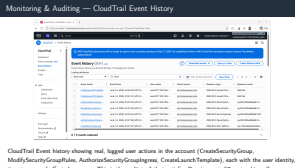
0 / 5 events selected

CloudTrail Event history showing real, logged user actions in the account (CreateSecurityGroup, ModifySecurityGroupRules, AuthorizeSecurityGroupIngress, CreateLaunchTemplate), each with the user identity,

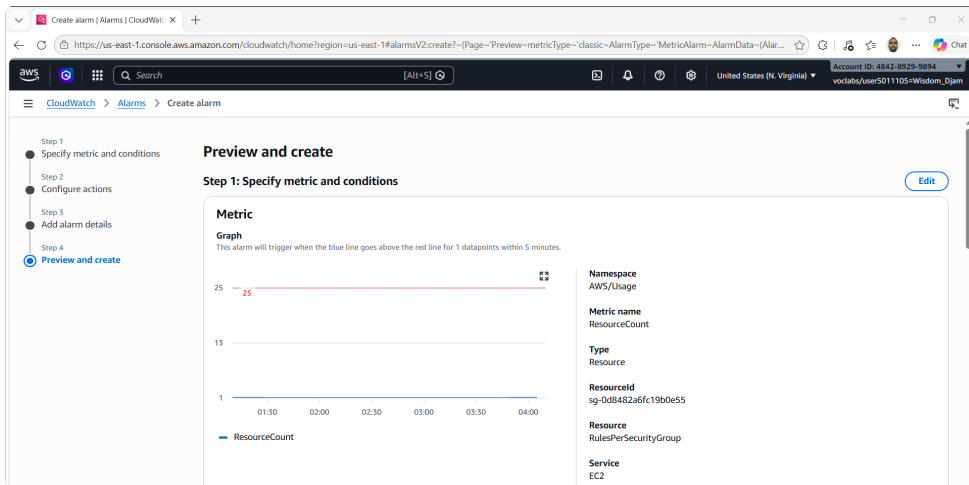
AWS Architecture: A Medical Company

Monitoring & Auditing — CloudTrail Event History

- This is a live demonstration of auditing: the events shown are my own actions from building the security groups, captured automatically by CloudTrail with identity and timestamp.
- It directly answers the customer's requirement to track all user activity, and Role-Play #5.



Monitoring & Auditing — CloudWatch Alarm



A CloudWatch alarm configured with a static threshold (greater than 25) over a 5-minute period. In the production design, a CPU-utilization alarm of this form is what triggers Auto Scaling to add or remove instances.

AWS Architecture: A Medical Company

└ Monitoring & Auditing — CloudWatch Alarm

- This shows the alarm fully configured: metric, threshold, and evaluation period.
- **Honest note:** the Learner Lab role lacks `cloudwatch:PutMetricAlarm`, so the alarm is shown configured rather than committed; in a production account it would be created and wired to the Auto Scaling policy.



What's next / further analysis

- **Proof of Concept:** deploy the Production VPC in one Region and load-test ELB + ASG self-healing and RDS Multi-AZ failover before launch.
- Formalize **HIPAA** alignment — sign the AWS BAA and confirm every PII-handling service is HIPAA-eligible.
- Add **multi-Region disaster recovery** and cross-Region S3 backup replication.
- **Cost optimization** pass — Reserved Instances / Savings Plans and right-sizing.
- Evaluate **Amazon Cognito** or federation for patient and doctor (end-user) authentication.

What I learned

- The real challenge was translating ambiguous **business requirements** into concrete AWS services — and communicating them to a non-technical customer — not the technology itself.
- Security, availability, and auditing are **cross-cutting** concerns: they shape every tier of the design, not a single section of it.

AWS Architecture: A Medical Company

└ Next Steps & Conclusion

Next Steps & Conclusion

What's next / further analysis

- **Proof of Concept:** deploy the Production VPC in one Region and load-test ELB + ASG self-healing and RDS Multi-AZ failover before launch.
- **Formalize HIPAA alignment** — sign the AWS BAA and confirm every PH-handling service is HIPAA-eligible.
- **Add multi-Region disaster recovery** and cross-Region S3 backup replication.
- **Cost optimization pass** — Reserved Instances / Savings Plans and right-sizing.
- **Evaluate Amazon Cognito** or federation for patient and doctor (end-user) authentication.

What I learned

- The real challenge was translating ambiguous business requirements into concrete AWS services — and communicating them to a non-technical customer — not the technology itself.
- Security, availability, and auditing are cross-cutting concerns: they shape every tier of the design, not a single section of it.

- **Recommend the POC and say what it proves:** “I’d recommend a proof of concept: stand up the Production VPC in one Region and load-test it to confirm the web and app tiers self-heal and the database fails over cleanly. That de-risks the launch.”
- **Show maturity by naming what I deliberately deferred:** multi-Region DR, cost tuning, and end-user authentication with Cognito are intentional fast-follows, not oversights — stating that signals good engineering judgment.
- **HIPAA close:** I’d formalize compliance with a signed BAA and a HIPAA-eligibility review before going live with real patient data.
- **Deliver the learning reflection genuinely:** the hardest part was requirements translation and explaining cloud concepts to a non-technical customer (the whole point of the role-play exercise), and the realization that

References

- Amazon Web Services. (n.d.). *AWS Well-Architected Framework*. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- Amazon Web Services. (n.d.). *What is Amazon VPC?* Retrieved June 16, 2026, from <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- Amazon Web Services. (n.d.). *Security best practices in IAM*. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- Amazon Web Services. (n.d.). *Amazon EC2 Auto Scaling user guide*. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/autoscaling/ec2/userguide/>
- Amazon Web Services. (n.d.). *Elastic Load Balancing*. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/>
- Amazon Web Services. (n.d.). *Multi-AZ deployments for high availability (Amazon RDS)*. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>
- Amazon Web Services. (n.d.). *What is AWS CloudTrail?* Retrieved June 16, 2026, from <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html>
- Amazon Web Services. (n.d.). *What is Amazon CloudWatch?* Retrieved June 16, 2026, from <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html>
- Amazon Web Services. (n.d.). *HIPAA compliance*. Retrieved June 16, 2026, from <https://aws.amazon.com/compliance/hipaa-compliance/>

Note: AWS documentation is continuously updated; entries use APA 7 “n.d.” with a retrieval date per the guidance for living web sources.

AWS Architecture: A Medical Company

References

References

• Amazon Web Services. (n.d.). AWS Well-Architected Framework. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
 • Amazon Web Services. (n.d.). What is Amazon VPC? Retrieved June 16, 2026, from <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-aws-vpc.html>
 • Amazon Web Services. (n.d.). Security best practices in IAM. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/IAM/latest/best-practices.html>
 • Amazon Web Services. (n.d.). Amazon EC2 Auto Scaling user guide. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/autoscaling/ec2/userguide/>
 • Amazon Web Services. (n.d.). Elastic Load Balancing. Retrieved June 16, 2026, from <https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/>
 • Amazon Web Services. (n.d.). Multi-AZ deployment for high availability (Amazon RDS). Retrieved June 16, 2026, from <https://docs.aws.amazon.com/amazondbs/latest/br/elastic/ha-multi-az.html>
 • Amazon Web Services. (n.d.). What is AWS CloudTrail? Retrieved June 16, 2026, from <https://docs.aws.amazon.com/cloudtrail/latest/userguide/cloudtrail-user-guide.html>
 • Amazon Web Services. (n.d.). What is Amazon CloudWatch? Retrieved June 16, 2026, from <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/what-is-cloudwatch.html>
 • Amazon Web Services. (n.d.). HIPAA compliance. Retrieved June 16, 2026, from <https://aws.amazon.com/compliance/hipaa-compliance/>
 Note: AWS documentation is continuously updated, verify use AWS T "n.d." with a retrieval date per the guidelines for citing web sources.

- **Purpose:** The paper guidance asks for APA references “where appropriate.” These are the AWS sources that ground the design’s factual claims (service capabilities and best practices), which also keeps me clear of the plagiarism check.
- **If asked:** I can speak to any one of these — e.g., the Well-Architected Framework’s five pillars shaped my whole approach; the IAM best-practices guide informed least privilege, MFA, and roles; the RDS Multi-AZ doc supports my database HA claim.
- **Before submitting:** i confirmed each URL resolves and replace “June 16, 2026” with the actual date I retrieved them. In the companion paper, these same sources became in-text citations (e.g., “CloudTrail records all account API activity (Amazon Web Services, n.d.)”).
- **This slide can be shown briefly or left as a reference.**

Questions?

Thank you — Wisdom Djam

- **Anticipated questions I should be ready to answer myself:**
- **Q: Why two VPCs instead of one with separate subnets?** Stronger isolation and a smaller blast radius — a problem in Dev/Test cannot reach Production PII.
- **Q: Why RDS instead of SQL Server on EC2?** Managed Multi-AZ failover, automated backups, and patching — less operational burden and stronger built-in HA. The customer needs no advanced RDBMS configuration, so managed is ideal.
- **Q: How does “double the servers” work?** It’s the Auto Scaling Group’s maximum capacity — a configuration value, so growth needs no new hardware.
- **Q: Is this HIPAA compliant?** I use HIPAA-eligible services and would sign an AWS BAA; full compliance is validated in the POC phase before any